

HEINRICH-HEINE-UNIVERSITÄT DÜSSELDORF

Bachelor Thesis

Robotic dog walking policies in the genesis simulator

Emre Gökcek

Degree:	B.Sc. Computer Science
Supervisor:	Professor Dr. Paul Swoboda
Co-Supervisor:	Peter Arndt
Faculty:	Faculty of Mathematics and Natural Sciences

13.06.2025

Abstract

We trained locomotion policies for the Unitree Go2 quadruped robot using deep reinforcement learning (RL) in the Genesis simulator. Specifically, we employed Proximal Policy Optimization (PPO) to solve three distinct tasks: performing a backflip, executing a frontflip, and navigating rough terrain at dynamically adjusted velocities. For each task, we systematically designed observation spaces, reward functions, and dynamics randomization strategies, and introduced curriculum learning to progressively increase task complexity. The experimental results demonstrated that carefully structured observations and rewards enabled the emergence of robust and dynamic locomotion behaviors. Notably, incorporating terrain height information significantly improved navigation performance on uneven surfaces. Detailed analyses of training dynamics, policy convergence, and generalization capabilities highlighted key insights for effective quadruped locomotion training in this new simulator.

1. Introduction

This study focused on training locomotion policies for the Unitree Go2 quadruped robot within the open-source Genesis simulator [1], which offers precise physics modeling and efficient parallelization for RL applications. Using PPO [2], we examined strategies tailored for quadruped locomotion. The key contributions are:

- A cohesive methodology for training policies in Genesis, covering the robot model, PD control, neural network design, and PPO implementation.
- An in-depth analysis of task-specific adjustments in observation spaces, reward structures, dynamics randomization, and curriculum learning, evaluating their effects on policy performance and robustness.
- Evidence that terrain height data in observations markedly improves navigation on uneven surfaces by enabling proactive adaptation to terrain variations.

2. Related Work

In this section, we discuss closely related works that employ deep RL for quadruped locomotion, focusing on methods, robot platforms, and training strategies relevant to our experiments.

Aractingi et al. [3] present a deep RL approach for controlling the Solo12 quadruped robot. Their method employs a neural network policy trained via PPO to output joint angle offsets from a nominal pose, similar to our approach. They utilize the Raisim [4] simulator for training and incorporate domain and dynamic randomization techniques, including randomization of PD controller gains and sensor noise, to facilitate sim-to-real transfer. Additionally, they introduce a curriculum learning strategy, gradually increasing the complexity of the reward function and terrain difficulty during training. Their reward function emphasizes velocity tracking, energy efficiency, and stability, penalizing deviations from desired joint configurations and excessive power consumption. The resulting policies demonstrate robust locomotion capabilities on diverse terrains, both indoors and outdoors.

Bellegarda et al. [5] propose a deep RL framework for high-speed running on the Unitree A1 quadruped robot. Their approach differs from ours by directly learning foot positions in Cartesian space, tracked via Cartesian PD control, rather than joint angle offsets. They employ PPO with an LSTM-based policy network to handle partial observability and improve sim-to-real transfer. Their training occur in PyBullet [6] and incorporate extensive dynamics randomization, including variations in robot mass, friction coefficients, and added payloads. The reward function is minimalistic, primarily rewarding velocity tracking and penalizing energy consumption and instability. Their method achieves robust high-speed running, demonstrating successful sim-to-real transfer with payloads up to 42% of the robot's nominal mass.

Sarrocco and Bertelli [7] provide a practical guide for training quadruped locomotion policies using deep RL in the Genesis simulator, specifically employing the Unitree Go2 robot model. Their approach closely aligns with ours, utilizing PPO to learn joint angle offsets from a default standing posture. They emphasize the importance of carefully designed observation spaces, including proprioceptive states and external velocity commands, and structured reward functions that balance velocity tracking, stability, and smoothness. Their training strategy incorporates domain randomization, varying parameters such as friction, mass distribution, and sensor noise to enhance robustness. Additionally, they simulated action latency to better approximate real-world conditions. Their resulting policies demonstrated effective locomotion and successful sim-to-real transfer.

This work builds upon these prior studies, adopting similar RL methodologies and training strategies. We investigate task-specific variations in observation spaces, reward structures, domain randomization, and curriculum learning strategies, providing an analysis of their impact on policy performance and robustness.

All of the code used in this work can be found at:

Github Link

The following section provides an overview of the general methodology used for all of the distinct tasks.

3. Method

3.1. Robot Model and Simulation Setup

All experiments utilized the Unitree Go2 quadruped robot model, loaded from a URDF file. The robot consists of 12 actuated degrees of freedom (DoFs), with three joints per leg: hip, thigh, and calf. Each joint was controlled via a proportional-derivative (PD) controller, defined by the control law:

$$\tau = k_p(\theta_{\text{target}} - \theta) - k_d\dot{\theta} \tag{1}$$

where τ denotes the joint torque, θ_{target} the target joint angle, θ the current joint angle, and $\dot{\theta}$ the joint angular velocity. The PD gains k_p and k_d were set per task and specified in the experiments section.

The robot was simulated using the Genesis simulator where the simulation timestep was set to $0.02\,\text{s}$, corresponding to a control frequency of $50\,\text{Hz}$. Each simulation step consisted of multiple physics substeps to ensure numerical stability and accuracy.

3.2. Markov Decision Process Formulation

Each locomotion task was formulated as a Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$, where:

- $\mathcal{S}$ is the state space, consisting of robot proprioceptive states (e.g., joint angles, velocities, base orientation, and velocities). The exact composition of the observation space varies per task and is detailed in the experiments section.
- $\mathcal{A}$ is the action space, defined as the desired joint angle offsets from a predefined default pose. Actions are continuous and bounded within [-100, 100].
- $\mathcal{P}$ represents the transition dynamics governed by the physics simulation.
- r is the reward function, task-specific and detailed in the experiments section.
- γ the discount factor, was set to 0.99 across all tasks.

3.3. Policy and Value Function Representation

We employed a neural network architecture consisting of two separate multilayer perceptrons (MLPs): an actor network representing the policy $\pi_\theta(a|s)$, and a critic network estimating the value function $V_\phi(s)$. Both networks shared a similar architecture with three hidden layers of sizes $[512, 256, 128]$ and ELU activation functions. The actor network outputs mean joint angle offsets, parameterizing a Gaussian distribution with a learned diagonal covariance matrix. The critic network outputs a scalar value estimate for the given state.

The policy's action noise standard deviation parameterizes the initial exploration behavior of the policy. A higher initial standard deviation ($\sigma = 1.0$) encourages broader exploration early in training, and this parameter was adapted during training.

3.4. PPO Setup

The policies were trained using the PPO implementation of the open-source library rsl_rl [8] with slight modifications such as curriculum learning (see 4.3.5).

The following hyperparameters were used for all tasks:

Hyperparameter	**Value**
Clipping parameter ϵ	0.2
GAE λ	0.95
Desired KL divergence	0.01
Learning rate	0.001 (adaptive)
Entropy coefficient	0.01
Value function loss coefficient	1.0
Gradient clipping norm	1.0
Number of epochs per update	5
Number of mini-batches per update	4
Number of steps per environment	24

Table 1. PPO Hyperparameters

The parameter *Number of epochs per update* specifies how many times the collected rollout data is reused for policy updates. Increasing this parameter allows the policy to learn more from the same data but may lead to overfitting.

The parameter *Number of steps per environment* determines how many simulation steps each parallel environment executes before performing a policy update. A higher value results in more data per update, potentially stabilizing training but increasing computational cost per update.

Training was parallelized across multiple simulation environments to accelerate data collection and improve sample efficiency. The exact number of parallel environments varied per task and is specified in the experiments section.

3.5. Adaptive Learning Rate Schedule

We employed an adaptive learning rate schedule based on the KL divergence between the old and new policy distributions. After each policy update, the mean KL divergence across the mini-batches is computed. If the mean KL divergence exceeds twice the desired threshold (0.01), the learning rate is decreased by a factor of 1.5. Conversely, if the mean KL divergence is less than half the desired threshold, the learning rate is increased by a factor of 1.5. The learning rate is bounded between 1×10^{-4} and 1×10^{-2}.

3.6. Action Latency Simulation

To better approximate real-world conditions, we simulated action latency by delaying the execution of policy actions by one simulation step. Specifically, at each simulation step, the actions executed by the robot were the actions computed at the previous step, rather than the current step. Formally, this was implemented as:

$$a_{\text{exec},t} = a_{t-1} \tag{2}$$

where $a_{\text{exec},t}$ denotes the action executed at timestep t, and a_{t-1} is the action computed by the policy at timestep $t-1$.

3.7. Task-Specific Variations

While the general methodology described above remains consistent across all tasks, certain aspects such as observation spaces, reward functions, dynamics randomization, curriculum learning strategies and length of episodes differed per task. These task-specific details are elaborated on in the following section.

4. Experiments

In this section, we present detailed experiments conducted to evaluate our methodology for training quadruped locomotion policies using PPO in the Genesis simulator.

4.1. Backflip Task

The backflip task involved training the Unitree Go2 quadruped robot to perform a dynamic backflip maneuver from a stationary standing position. The code for this task was mainly inspired by [9].

4.1.1 Robot Model and PD Control Setup

Each environment started with the same default standing joint angles as described in the following table 2.

Leg	Hip	Thigh	Calf
Front Left (FL)	0	0.8	-1.5
Front Right (FR)	0	0.8	-1.5
Rear Left (RL)	0	1.0	-1.5
Rear Right (RR)	0	1.0	-1.5

Table 2. Default Standing Joint Angles (rad)

Each joint was controlled via a PD controller with nominal gains $k_p = 70\,\text{Nm/rad}$ and $k_d = 3\,\text{Nms/rad}$. The target joint angles were computed as:

$$\theta_{\text{target}} = \theta_{\text{default}} + 0.5\,a$$

where $a \in [-100, 100]^{12}$ is the clipped action vector output by the policy. Torque was applied at a control frequency of $50\,\text{Hz}$ with a physics timestep of $0.005\,\text{s}$ (4 substeps per control step). Action latency of one timestep was simulated.

4.1.2 Simulation and Episode Setup

Training was conducted in headless mode. The environment consisted of a flat plane without terrain variations. Each episode lasted exactly $0.02\,\text{s}$ (100 control steps), with no early termination conditions, so an environment was only reset once it reached 100 steps.

4.1.3 Observation Space

The observation vector $s_t \in \mathbb{R}^{60}$ comprises scaled components to provide the policy with relevant state information while normalizing inputs for stable learning. It includes the base angular velocity (scaled by 0.25 to map typical values of ±4 rad/s to approximately ±1), the projected gravity vector, deviations of joint angles from their default pose, joint velocities (scaled by 0.05 to normalize values up to ±20 rad/s), the current action, the previous action, and sinusoidal terms derived from a phase variable to encode temporal progression. These scalings ensure that most observation elements fall within a roughly [-1, 1] range, promoting consistent gradient flow in the neural network. The phase term, denoted as ϕ, was defined as $\phi = \frac{\pi}{2}t$, where t is the elapsed time in seconds since the episode started. This formulation makes ϕ increase linearly from 0 to π over the episode's duration of 2 seconds, providing the policy with an explicit representation of temporal context. To capture temporal patterns at various scales—such as the rapid push-off phase and the slower rotational phase of a backflip—the observation included multiple sinusoidal transformations: $\sin(\phi)$, $\cos(\phi)$, $\sin(\frac{\phi}{2})$, $\cos(\frac{\phi}{2})$, $\sin(\frac{\phi}{4})$, and $\cos(\frac{\phi}{4})$. The detailed composition is provided in Table 3.

Description	**Dimension**
Base angular velocity (ω_b, scaled by 0.25)	3
Projected gravity vector (g_b)	3
Joint angle deviations ($\theta - \theta_{\text{default}}$)	12
Joint velocities ($\dot{\theta}$, scaled by 0.05)	12
Current action (a_t)	12
Previous action (a_{t-1})	12
Sinusoidal phase terms	6

Table 3. Observation Space Composition

Additionally, a 64-dimensional privileged observation vector (available only to the critic) incorporates the base height, base linear velocity (scaled by 2.0 to align typical

magnitudes of $\pm 0.5\,\mathrm{m/s}$ with other inputs), and the same proprioceptive and phase terms.

4.1.4 Reward Function

The reward function is a weighted sum of multiple components designed to encourage a successful backflip maneuver. The total reward at each timestep t is given by:

$$r_t = \sum_i w_i r_{i,t}$$

where w_i is the weight for the i-th reward component $r_{i,t}$. The individual components are defined as follows:

Angular Velocity Y Reward Encourages a specific angular velocity around the y-axis during the flip phase, active between 0.5 and 1.0 seconds.

$$r_{\text{ang_vel_y}} = -\text{clamp}(\omega_y, -7.2, 7.2) \cdot 1_{\{0.5<t<1.0\}}$$

where ω_y is the angular velocity around the y-axis (rad/s), and t is the time (seconds).

Linear Velocity Z Reward Encourages upward linear velocity during the initial phase, active between 0.5 and 0.75 seconds.

$$r_{\text{lin_vel_z}} = \text{clamp}(v_z, -\infty, 3) \cdot 1_{\{0.5<t<0.75\}}$$

where v_z is the linear velocity along the z-axis (m/s).

Angular Velocity Z Penalty Penalizes angular velocity around the z-axis to maintain stability.

$$r_{\text{ang_vel_z}} = |\omega_z|$$

where ω_z is the angular velocity around the z-axis (rad/s), and t is the time (seconds).

Orientation Control Penalty Penalizes deviations from a desired time-dependent orientation.

$$r_{\text{orientation}} = \|\mathbf{g}_{\text{proj}} - \mathbf{g}_{\text{des}}\|^2$$

where $\mathbf{g}_{\text{proj}}$ is the current projected gravity vector in the robot's base frame and $\mathbf{g}_{\text{des}}$ is the desired projected gravity vector, computed as $\mathbf{g}_{\text{des}} = \mathbf{R}_{\text{des}}^T \mathbf{g}$. The desired rotation matrix $\mathbf{R}_{\text{des}}$ corresponds to a time-dependent pitch rotation: $\theta_{\text{pitch}}(t) = 4\phi(t)\pi$, where $\phi(t) = \text{clamp}(t - 0.5, 0, 0.5)$ is the phase parameter that drives a full 2π backflip rotation over the interval $t \in [0.5, 1.0]$ seconds, and $\mathbf{g} = [0, 0, -9.81]^T$ m/s^2 is the global gravity vector.

Feet Height Before Backflip Penalty Penalizes height of feet above $0.02\,\mathrm{m}$ before the backflip, active for $t < 0.5$.

$$r_{\text{feet_height}} = \sum_{k=1}^{4} \max(z_k - 0.02, 0) \cdot 1_{\{t<0.5\}}$$

where z_k is the z-coordinate of the k-th foot (m).

Height Control Penalty Penalizes deviations from a target height of $0.3\,\mathrm{m}$ when $t < 0.4$ or $t > 1.4$. This is to ensure that the robot starts and ends the flip at the same position.

$$r_{\text{height_control}} = (h - 0.3)^2 \cdot (1_{\{t<0.4\}} + 1_{\{t>1.4\}})$$

where h is the height of the robot's base (m).

Actions Symmetry Penalty This penalty enforces coordinated joint actions for a balanced backflip by promoting symmetry and anti-symmetry between corresponding joints of the quadruped robot's legs. The hip sums ($(a_0 + a_3)^2$ and $(a_6 + a_9)^2$) encourage anti-symmetry to stabilize lateral hip movements, while the thigh/calf differences ($\sum_{i=1}^{2}(a_i - a_{i+3})^2$ and $\sum_{i=7}^{8}(a_i - a_{i+3})^2$) enforce symmetry for synchronized leg movements. This promotes a stable, coordinated backflip.

$$\begin{aligned} r_{\text{actions_symmetry}} =& (a_0 + a_3)^2 + \sum_{i=1}^{2}(a_i - a_{i+3})^2 \\ &+ (a_6 + a_9)^2 + \sum_{i=7}^{8}(a_i - a_{i+3})^2 \end{aligned}$$

where a_i is the control action for the i-th degree of freedom.

Gravity Y Penalty Penalizes the y-component of the projected gravity vector, related to roll orientation.

$$r_{\text{gravity_y}} = (g_{\text{proj},y})^2$$

Feet Distance Penalty Penalizes deviations from a desired stance width in the y-direction.

$$r_{\text{feet_distance}} = \sum_{k=1}^{4}(y_k - d_k)^2$$

where y_k is the y-position of the k-th foot in the body frame (m), and $d_k = 0.15$ for $k = 0, 2$, $d_k = -0.15$ for $k = 1, 3$.

Action Rate Penalty Penalizes rapid changes in actions to encourage smooth control.

$$r_{\text{action_rate}} = \sum_{i=1}^{12}(a_i^{(t)} - a_i^{(t-1)})^2$$

where $a_i^{(t)}$ is the action at time t.

The weights for each reward and penalty term are summarized in Table 4. Note that penalty terms are assigned negative weights.

$\mathbf{w}_{\text{pitch}}$	5.0
$\mathbf{w}_{\text{yaw}}$	-1.0
$\mathbf{w}_{\text{lin_vel_z}}$	20.0
$\mathbf{w}_{\text{orientation}}$	-1.0
$\mathbf{w}_{\text{feet_height}}$	-30.0
$\mathbf{w}_{\text{height_control}}$	-10.0
$\mathbf{w}_{\text{actions_symmetry}}$	-0.1
$\mathbf{w}_{\text{gravity_y}}$	-10.0
$\mathbf{w}_{\text{feet_distance}}$	-1.0
$\mathbf{w}_{\text{action_rate}}$	-0.001

Table 4. Reward weights

4.1.5 Dynamics Randomization

To enhance robustness and facilitate sim-to-real transfer, dynamics randomization [10] was applied at each environment reset. Randomized parameters and their ranges are summarized in Table 5.

Parameter	Range
Link friction coefficient	[0.2, 1.5]
Added mass on base link	[-1 kg, +3 kg]
COM shift on base link (x/y/z)	[-0.01, 0.01] m
Joint offset bias	[-0.02, 0.02] rad
PD gain scale (k_p, k_d)	[0.8, 1.2]

Table 5. Dynamics Randomization Parameters

4.1.6 Training Setup

The policy was trained with 10,000 parallel environments for 1000 iterations and training converged after 400 iterations which took approximately 17 minutes on a single RTX-4090 GPU. PPO hyperparameters were identical to those described in Section 3.

4.1.7 Results and Analysis

The trained policy successfully learned to perform a dynamic backflip maneuver from a stationary standing position. The robot demonstrates stable takeoff, controlled rotation, and a balanced landing, consistently achieving the desired maneuver. A representative video demonstrating the evaluated policy performing the backflip can be found here: Video File.

Several key metrics were logged throughout training, as illustrated in Figure 1. The mean episode reward rapidly increases during the initial training phase, stabilizing after approximately 400 PPO iterations. This indicates successful convergence of the policy, as the robot consistently achieves high cumulative rewards.

The individual reward terms provide further insights into the learning dynamics. The angular velocity reward around the y-axis quickly rises and stabilizes, indicating that the policy effectively learns to generate and maintain the desired rotational velocity necessary for the backflip. Similarly, the linear velocity reward along the z-axis increases sharply during early training, reflecting the policy's ability to produce sufficient upward momentum for a successful takeoff.

The value function loss decreases rapidly during the initial training phase and remains close to zero after approximately 400 iterations, suggesting that the critic network accurately estimates the expected returns, contributing to stable policy updates. The surrogate loss exhibits initial fluctuations but stabilizes around zero, indicating that the PPO updates remain within the desired trust region, thus ensuring stable and controlled policy improvements. Additionally, the mean noise standard deviation steadily decreases throughout training, reflecting a reduction in exploration as the policy converges to an optimal solution.

We also experimented with training using fewer parallel environments (4096 instead of 10,000). However, this configuration did not yield a converged policy. This outcome suggests that the short episode length (2 seconds) limits the amount of experience collected per iteration, necessitating a larger number of parallel environments to ensure sufficient data for stable policy updates.

4.2. Frontflip Task

The frontflip task involved training the Unitree Go2 quadruped robot to perform a dynamic frontflip maneuver from a stationary standing position. Similar to the backflip task, this maneuver demanded precise coordination, accurate timing, and robust control to achieve a successful flip and stable landing. The experimental setup, including the robot model, PD control setup, simulation parameters, observation space, and dynamics randomization, remains identical to the backflip task described in Section 4.1. In this subsection, we highlight only the differences in the reward function formulation and training details specific to the frontflip task.

4.2.1 Reward Function Differences

The reward function for the frontflip task closely mirrors that of the backflip task, with the following modifications:

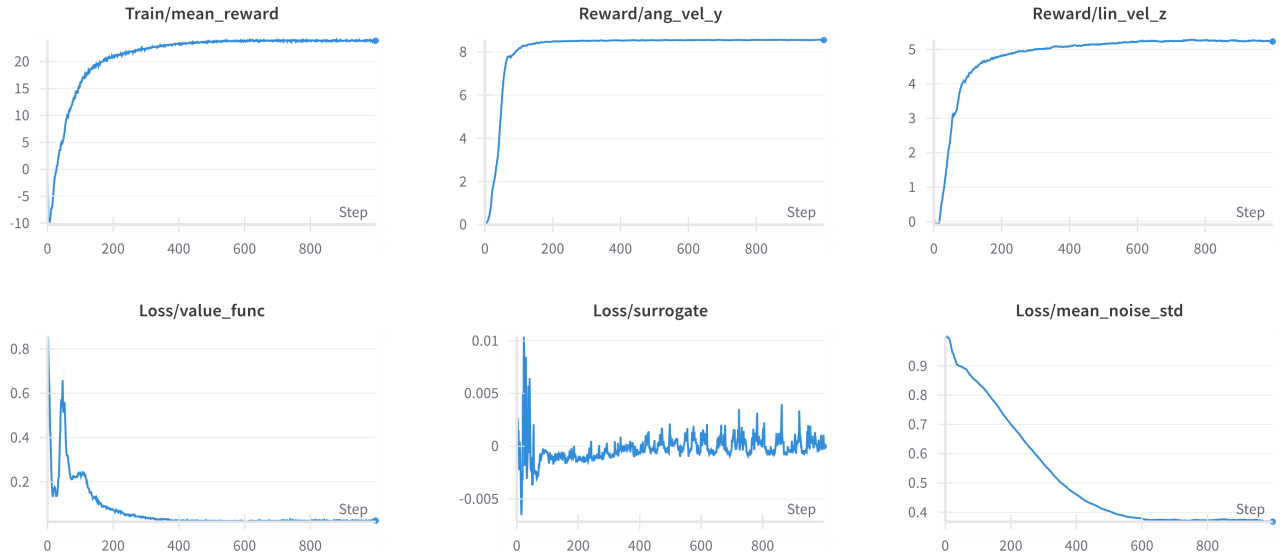


Figure 1. Training metrics logged during the backflip task.

Angular Velocity Y Reward In contrast to the backflip task, the angular velocity around the y-axis is positively rewarded during the flip phase to encourage forward motion, active between 0.5 and 1.0 seconds:

$$r_{\text{ang_vel_y}} = w_{\text{ang_vel_y}} \cdot \big(\text{clamp}(\omega_y, -10, 10) \cdot 1_{\{0.5<t<1.0\}}\big)$$

Also, the clamped values were increased to [-10, 10], as higher velocity was needed to perform the motion.

Orientation Control Penalty The desired orientation trajectory is inverted to reflect the forward rotation of a frontflip. Specifically, the desired pitch angle trajectory is defined as:

$$\theta_{\text{pitch}}(t) = -4\phi(t)\pi, \quad \phi(t) = \text{clamp}(t - 0.5, 0, 0.5)$$

Collision Penalty An additional collision penalty is introduced to discourage undesirable contacts between the robot's base, thighs, or calves and the ground:

$$r_{\text{collision}} = w_{\text{collision}} \cdot \sum_{i \in \mathcal{C}} \mathbf{1}\{\|\mathbf{f}_i\| > 0.1\}$$

where $\mathcal{C}$ denotes the set of penalized links (*base*, *thigh*, *calf*), and $\mathbf{f}_i$ represents the contact force vector acting on link i.

The adjusted reward weights for the frontflip task are summarized in Table 6:

$\mathbf{w}_{\text{ang_vel_y}}$	7.0
$\mathbf{w}_{\text{ang_vel_z}}$	-1.0
$\mathbf{w}_{\text{lin_vel_z}}$	30.0
$\mathbf{w}_{\text{orientation_control}}$	-1.0
$\mathbf{w}_{\text{feet_height_before_frontflip}}$	-30.0
$\mathbf{w}_{\text{height_control}}$	-15.0
$\mathbf{w}_{\text{actions_symmetry}}$	-0.1
$\mathbf{w}_{\text{gravity_y}}$	-10.0
$\mathbf{w}_{\text{feet_distance}}$	-1.0
$\mathbf{w}_{\text{action_rate}}$	-0.01
$\mathbf{w}_{\text{collision}}$	-10.0

Table 6. Reward Weights for Frontflip Task

4.2.2 Training Setup and Results

We trained the frontflip policy using 10,000 parallel environments for 1200 PPO iterations, observing reasonable convergence after approximately 700 iterations, compared to 400 iterations required for the backflip task. This increased training duration suggests a higher inherent difficulty of the frontflip maneuver, or a suboptimal reward structure.

The resulting policy successfully performs the frontflip maneuver, demonstrating stable takeoff, controlled rotation, and balanced landing phases. A representative video demonstrating the evaluated policy performing the frontflip can be found here:

Video File

Key metrics were logged during training, as shown in Figure 2. When compared to the metrics from the backflip task (Figure 1), distinct differences are evident.

The angular velocity around the y-axis reward is nearly twice as high, which can be attributed to the increased weight and higher clamp value assigned to this reward, consequently resulting in a mean reward that is also approximately doubled. Additionally, the value loss converges more slowly, likely due to the greater complexity of the frontflip task. This complexity may also contribute to the slower reduction in noise standard deviation, as the agent requires a longer exploration period.

4.3. Rough Terrain Navigation Task

In this task, we trained the Unitree Go2 quadruped robot to navigate uneven terrain fast and robustly. Specifically, the fractal terrain provided by the Genesis simulator was utilized, characterized by irregular height variations and complex surface profiles. To systematically enhance the robot's locomotion capabilities, we employed a curriculum learning strategy, progressively increasing the commanded forward velocity based on the robot's performance. The details of the curriculum learning approach is detailed in Section 4.3.5.

4.3.1 PD Control Setup

For this task, we set the PD gains to $k_p = 20\,\text{Nm/rad}$ and $k_d = 1\,\text{Nms/rad}$. The target joint angles were computed as offsets from a predefined default standing pose, scaled by an action scaling factor of 0.25:

$$\theta_{\text{target}} = \theta_{\text{default}} + 0.25 \cdot a$$

The default standing joint angles, consistent across all tasks, are summarized in Table 2.

4.3.2 Simulation and Episode Setup

The simulation environment consisted of fractal terrain generated by the Genesis simulator, characterized by irregular height variations and complex surface profiles. Each terrain tile measured $12\,\text{m} \times 12\,\text{m}$. Four terrain tiles were arranged sequentially, forming a continuous terrain of $48\,\text{m}$ length. Each simulation episode lasted a maximum of $30\,\text{s}$, corresponding to 1500 control steps at a control frequency of $50\,\text{Hz}$. Episodes terminated prematurely if the robot's roll exceeded $20°$, pitch exceeded $10°$, or if the robot's base contacted the terrain. Action latency of one timestep was simulated to approximate real-world actuator delays.

4.3.3 Observation Space

The observation vector $s_t \in \mathbb{R}^{73}$ comprises scaled components to provide the policy with relevant state information while normalizing inputs for stable learning. The detailed composition is provided in Table 7.

Description	Dimension
Base linear velocity (scaled by 2.0)	3
Base angular velocity (scaled by 0.25)	3
Projected gravity vector	3
Joint angle deviations ($\theta - \theta_{\text{default}}$)	12
Joint velocities (scaled by 0.05)	12
Current actions	12
Base position change	3
Relative terrain heights	25

Table 7. Observation Space Composition for Rough Terrain Navigation Task

Relative Terrain Height Calculation To enable the robot to anticipate and adapt to upcoming terrain variations, we compute a local height map relative to the robot's current base position. Specifically, we define a 5×5 grid of points spaced evenly in front of the robot, covering an area of $1.6\,\text{m}$ in length (x-direction) and $1.6\,\text{m}$ in width (y-direction). The grid points are initially defined in the robot's local coordinate frame and subsequently transformed into the global coordinate frame using the robot's current orientation and position:

- **Local Grid Definition:** A 5×5 grid of points is defined in the robot's local frame, with spacing of $0.4\,\text{m}$ between points in both x and y directions.
- **Rotation to Global Frame:** Each local grid point is rotated into the global frame using the robot's current orientation quaternion.
- **Translation to Global Position:** The rotated points are translated by the robot's current global position, yielding global coordinates for each grid point.
- **Terrain Height Sampling:** The global coordinates are mapped onto the terrain height field grid, and the corresponding terrain heights are sampled.
- **Relative Height Computation:** The sampled terrain heights are subtracted from the robot's current base height, resulting in relative height values indicating the terrain elevation relative to the robot's base.

Additionally, a 97-dimensional privileged observation vector (available only to the critic) also includes the previous joint velocities (12 dimensions) and the previous actions issued by the policy (12 dimensions).

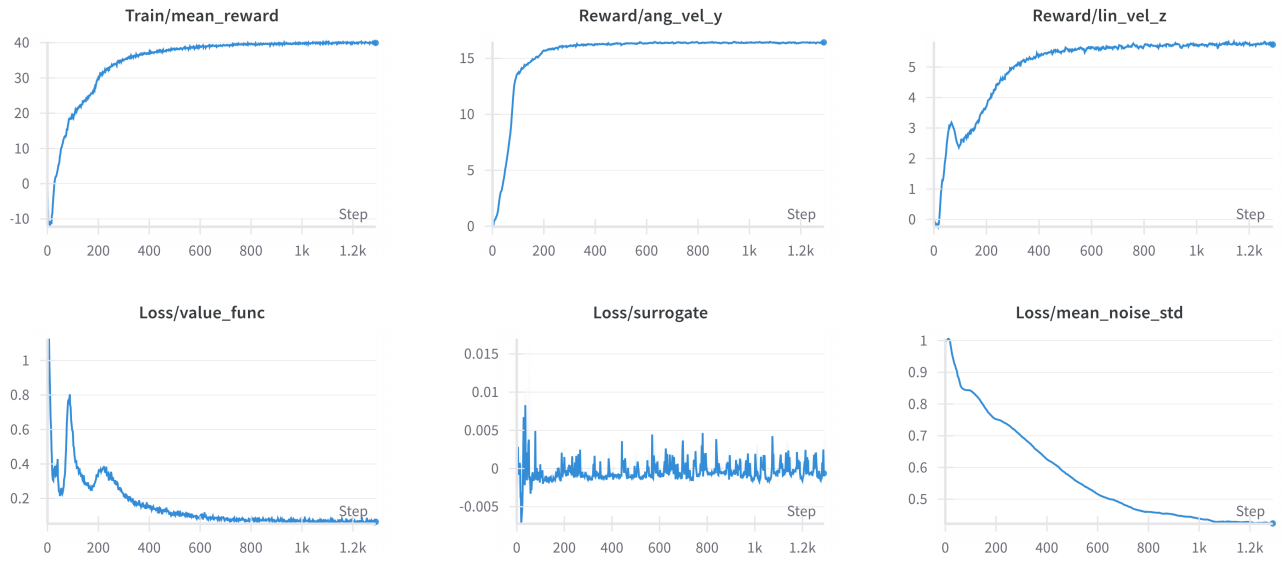


Figure 2. Training metrics logged during the frontflip task.

4.3.4 Reward Function Setup

The reward function for the rough terrain navigation task is structured to encourage precise tracking of commanded velocities while penalizing undesirable behaviors such as excessive lateral or vertical movements, and inefficient joint usage. The total reward is defined as:

$$r_t = \sum_i w_i r_{i,t}$$

where w_i is the weight for the i-th reward component $r_{i,t}$.

The individual components are defined as follows:

Linear Velocity Tracking Reward Encourages the robot to accurately track the commanded forward velocity $v_{\text{cmd},x}$. An exponential function is used to smoothly map velocity tracking errors to a reward value between 0 and 1, providing a continuous and differentiable gradient for stable learning:

$$r_{\text{track_vel_x}} = w_{\text{track_vel_x}} \cdot \exp\left(-\frac{(v_{\text{cmd},x} - v_x)^2}{\sigma}\right)$$

where v_x is the robot's current forward velocity, $\sigma = 0.3$, and with weight $w_{\text{track_vel_x}}$.

Angular Velocity Tracking Reward Encourages the robot to accurately track the commanded angular velocity around the vertical axis $\omega_{\text{cmd},z}$. Similarly, an exponential function ensures smooth reward transitions:

$$r_{\text{track_ang_vel}} = w_{\text{track_ang_vel}} \cdot \exp\left(-\frac{|\omega_{\text{cmd},z} - \omega_z|}{\sigma}\right)$$

where ω_z is the robot's current angular velocity around the vertical axis, $\sigma = 0.3$, $\omega_{\text{cmd},z} = 0$, and with weight $w_{\text{track_ang_vel}}$.

Vertical Velocity Penalty Penalizes vertical velocity to discourage unnecessary vertical movements:

$$r_{\text{lin_vel_z}} = w_{\text{lin_vel_z}} \cdot v_z^2$$

where v_z is the robot's vertical velocity, and with weight $w_{\text{lin_vel_z}}$.

Lateral Velocity Penalty Penalizes lateral velocity to discourage sideways movements:

$$r_{\text{lin_vel_y}} = w_{\text{lin_vel_y}} \cdot v_y^2$$

where v_y is the robot's lateral velocity, with weight $w_{\text{lin_vel_y}}$.

Action Rate Penalty Penalizes rapid changes in actions to encourage smooth control:

$$r_{\text{action_rate}} = w_{\text{action_rate}} \cdot \sum_{i=1}^{12} (a_i^{(t)} - a_i^{(t-1)})^2$$

where $a_i^{(t)}$ is the action at timestep t, with weight $w_{\text{action_rate}}$.

Default Pose Similarity Penalty Penalizes deviations from the default standing joint angles to encourage

stable and energy-efficient postures:

$$r_{\text{similar_to_default}} = w_{\text{similar_to_default}} \cdot \sum_{i=1}^{12} |\theta_i - \theta_{\text{default},i}|$$

where θ_i is the current joint angle and $\theta_{\text{default},i}$ is the default standing joint angle for joint i, and with weight $w_{\text{similar_to_default}}$.

Sideway Movement Penalty Penalizes lateral displacement from the initial starting position to encourage straight-line locomotion:

$$r_{\text{sideway_movement}} = w_{\text{sideway_movement}} \cdot \min(|y - y_{\text{init}}|, 2.0)$$

where y is the robot's current lateral position and y_{init} is the initial lateral position at episode start. The penalty is capped at 2.0 to prevent excessive penalization, and with weight $w_{\text{sideway_movement}}$.

The weights for each reward and penalty term are summarized in Table 8:

$w_{\text{track_vel_x}}$	1.0
$w_{\text{track_ang_vel}}$	1.0
$w_{\text{lin_vel_z}}$	-1.0
$w_{\text{lin_vel_y}}$	-5.0
$w_{\text{action_rate}}$	-0.005
$w_{\text{similar_to_default}}$	-0.1
$w_{\text{sideway_movement}}$	-1.0

Table 8. Reward Weights for Rough Terrain Navigation Task

4.3.5 Curriculum Learning Approach

Directly training the robot to navigate rough terrain at a target velocity like 1.0 m/s proved challenging, as the complexity of the task often led the policy to adopt overly conservative behaviors, such as standing still, to avoid penalties. To address this issue, we employed a curriculum learning strategy, progressively increasing the difficulty of the task by incrementally raising the commanded forward velocity based on the robot's performance.

Initially, we set the commanded linear velocity in the forward direction to a moderate value of 0.4 m/s. We tracked the mean of the Linear Velocity Tracking Reward over the latest 20 PPO iterations. Whenever this mean reward reached or exceeded a predefined curriculum threshold of 0.85—a value determined empirically through experimentation—we increased the commanded velocity by 0.05 m/s simultaneously across all parallel environments.

To further ensure consistent progression and mitigate potential stagnation due to reward fluctuations or noise, we introduced an additional criterion: if 500 PPO iterations elapsed without meeting the reward-based threshold, we automatically increased the commanded velocity by 0.05 m/s.

4.3.6 Training Setup

We trained the policy for a total of 3000 PPO iterations using 4096 parallel simulation environments. Due to the significantly longer episode length compared to the backflip task, fewer parallel environments were sufficient to collect adequate experience for stable policy updates. Training was conducted on a single RTX-4090 GPU and required approximately 50 minutes to reach convergence. The PPO hyperparameters used were identical to those described previously in Section 3.

4.3.7 Results and Analysis

With this training and curriculum setup, the model achieved a constant velocity of 1 m/s on fractal terrain, resulting in 12 velocity increments (each by 0.05 m/s) through the curriculum. The model maintained a straight and smooth gait with minimal lateral drift. Before incorporating relative height difference information into the observation space (see 4.3.3), the model struggled with sudden higher elevations in the terrain, often resulting in falls. This demonstrates that providing the policy network with terrain height data immediately in front of the agent significantly improves its ability to adapt and navigate varying terrain effectively. Additionally, prior to introducing the sideways movement penalty, the robot could navigate the terrain at the desired speed but exhibited significant lateral drift, failing to maintain a straight path and oscillating between left and right movements. Separately, it is observed that no explicit rewards or penalties targeting foot placement or stepping patterns were necessary to develop a functional gait, unlike the approach described in [3]. This may result from the policy not being evaluated in real-world conditions, which merits further investigation. Videos of the evaluated policy from different angles are available at:

Front View, Side View

Also, several metrics were logged during training, as presented in Figure 3.

The variations in mean reward are attributable to the curriculum approach, where the agent must adapt to each new target velocity before stabilizing. The marked reduction in lateral drift after initial declines reflects the model's progressive learning of a stable, straight gait pattern as training advances. However, the consistent rise in mean noise standard deviation after approximately step 2000 suggests ongoing exploration by the model, despite the emergence of a stable gait pattern around step 2800.

Additionally, we evaluated the policy on various terrains available in the Genesis simulator that were not part of the

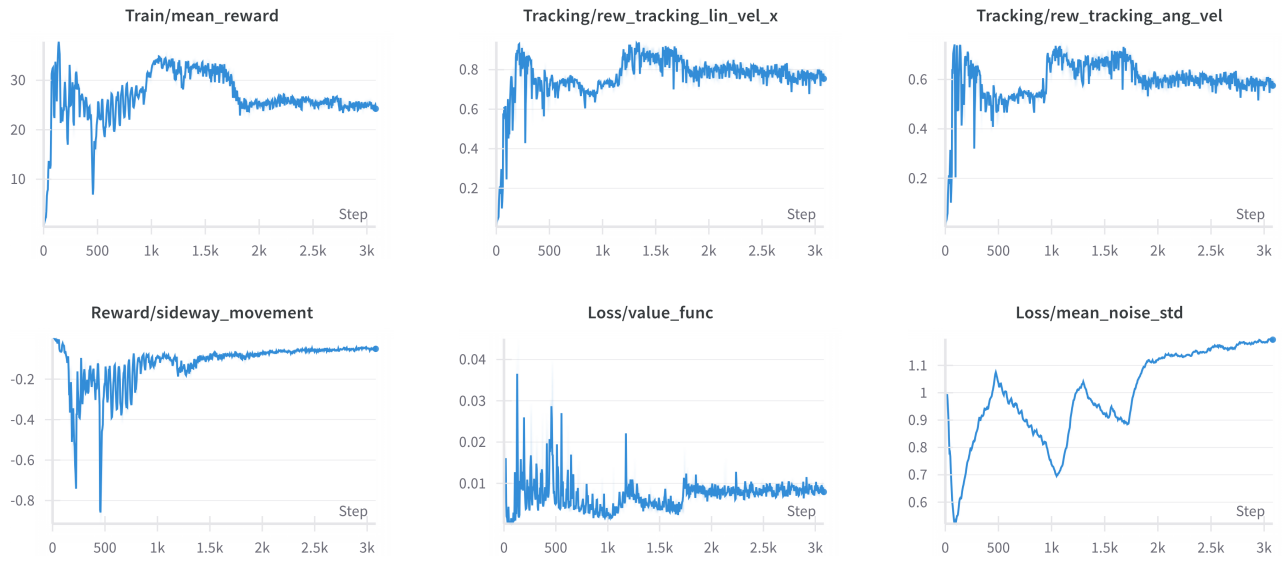


Figure 3. Training metrics logged during the rough terrain navigation task.

training set to assess its generalization capability. A video demonstrating the policy navigating four unseen terrains is available at Video File. The results show that the policy effectively generalizes to unfamiliar terrain, maintaining its speed and a stable gait, at least in the context of the simulator.

5. Conclusion

In this work, we trained locomotion policies for the Unitree Go2 quadruped robot using PPO in the Genesis simulator, focusing on dynamic maneuvers and robust terrain navigation. Incorporating relative terrain height information into the observation space significantly enhanced the robot's ability to anticipate and adapt to complex terrain profiles. Additionally, stable and efficient gaits emerged naturally from carefully balanced velocity tracking and stability penalties, without explicit gait-related rewards. Future research could explore embedding commanded velocities as learnable inputs within the policy network, potentially enabling seamless generalization across a continuous range of velocities without manual curriculum adjustments. Further investigation into domain randomization strategies, such as varying terrain characteristics between episodes, could also enhance policy robustness. Additionally, transitioning from simulator-derived terrain information to purely visual inputs from onboard cameras would present an interesting and challenging extension, requiring the policy to implicitly infer terrain characteristics from visual perception alone. Most importantly, deploying and evaluating these learned policies on a physical Unitree Go2 robot would provide critical insights into the fidelity of the Genesis simulator, highlight existing sim-to-real gaps, and guide future improvements.

References

[1] "Genesis: A universal and generative physics engine for robotics and beyond," December 2024. [Online]. Available: https://github.com/Genesis-Embodied-AI/Genesis 1

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *CoRR*, vol. abs/1707.06347, 2017. [Online]. Available: http://arxiv.org/abs/1707.06347 1

[3] M. Aractingi, P.-A. Léziart, T. Flayols, J. Perez, T. Silander, and P. Souéres, "Controlling the solo12 quadruped robot with deep reinforcement learning," *Scientific Reports*, vol. 13, no. 1, 2023. [Online]. Available: https://doi.org/10.1038/s41598-023-38259-7 1, 9

[4] J. Hwangbo, J. Lee, and M. Hutter, "Per-contact iteration method for solving contact dynamics," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 895–902, 2018. [Online]. Available: www.raisim.com 1

[5] G. Bellegarda, Y. Chen, Z. Liu, and Q. Nguyen, "Robust high-speed running for quadruped robots via deep reinforcement learning," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 10 364–10 370. [Online]. Available: https://doi.org/10.1109/IROS47612.2022.9982132 1

[6] E. Coumans and Y.Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning." [Online]. Available: https://pybullet.org/ 1

[7] F. Sarrocco and L. Bertelli, "Making quadrupeds learning to walk: Step-by-step guide," February 2025. [Online]. Available: https://federicosarrocco.com/blog/Making-Quadrupeds-Learning-To-Walk 1

[8] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164. PMLR, 2022, pp. 91–100. [Online]. Available: https://proceedings.mlr.press/v164/rudin22a.html 2

[9] "Genesis-backflip," December 2024. [Online]. Available: https://github.com/ziyanx02/Genesis-backflip 3

[10] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2018, p. 3803–3810. [Online]. Available: http://dx.doi.org/10.1109/ICRA.2018.8460528 5

Erklärung

Hiermit versichere ich, dass ich die Bachelorarbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Düsseldorf, den 13.06.2025

Emre Gökcek